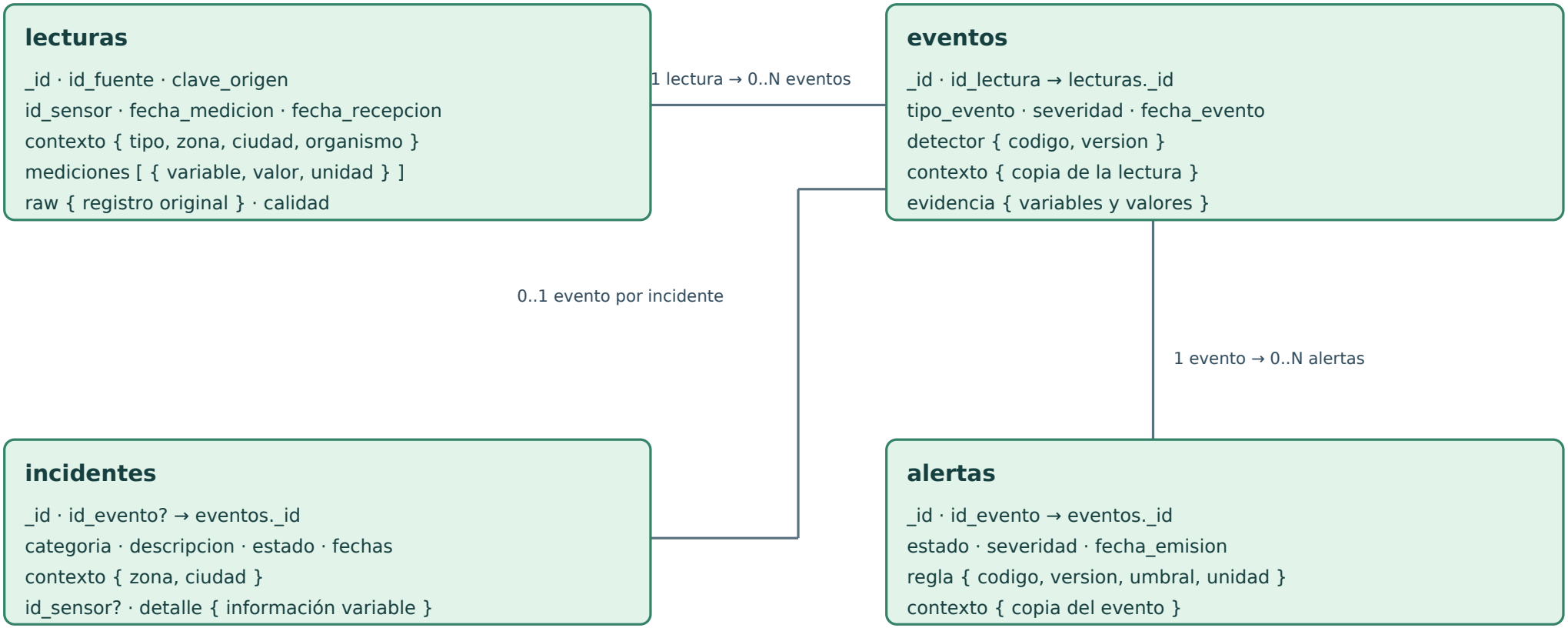
01 / Diagrama documental

Las cajas son colecciones y objetos embebidos. Las líneas entre colecciones son referencias lógicas, no claves foráneas automáticas.



Los IDs y el contexto se resuelven desde PostgreSQL al ingresar cada documento. No se agregan arrays crecientes de lecturas, eventos o alertas dentro de un sensor.

PostgreSQL: catálogos autoritativos

sensores · tipos · categorías · ubicaciones · ciudades · organismos · fuentes · variables · rutas · estacionamientos

02 / Contrato común e identificadores

Base propuesta: smartcity. Cuatro colecciones normales; no se requiere una colección independiente por sensor o por variable.

Campo	Tipo BSON / regla	Aplicación
_id	string no vacía; clave única e inmutable	Todas. ID estable generado por la ingesta. Se usan prefijos lec:, evt:, ale: e inc:.
schema_version	int = 1	Todas; permite evolucionar el esquema con migraciones explícitas.
id_fuente	long positivo	Todas. Mismo ID del catálogo PostgreSQL; identifica una versión del dataset.
clave_origen	string no vacía	Todas. Clave estable por colección y fuente; incluye tipo de derivación si aplica.
naturaleza	real | sintetica | no_verificada	Todas. Se hereda de la fuente; no se presenta una simulación como observación real.
procedencia	original | derivado | simulado	Todas. Describe cómo se produjo el documento, no la naturaleza de su fuente.
contexto	object	Todas. id_ubicacion e id_ciudad obligatorios. nombre_zona opcional; copia de catálogo.
contexto del sensor	id_tipo_sensor, id_categoria: long; id_organismo?: long	Obligatorio en lecturas/eventos/alertas. En incidentes, solo si existe id_sensor.
id_sensor	long positivo	Obligatorio en lecturas/eventos/alertas; opcional en incidentes.
Fechas y medidas	date; long para IDs; decimal para valores	Fechas UTC; conservar zona de origen documentada. No mezclar strings con números.

Referencia a PostgreSQL. Se adopta el nombre lógico id_tipo_sensor aunque el conceptual lo abrevie como id_tipo. Los bigint de PostgreSQL se representan como BSON long para evitar pérdida de precisión. El _id documental es string y se copia a lecturas_limpias.id_documento_mongo; id_lectura de PostgreSQL se genera por separado.

03 / Diseño de lecturas

Una fila del dataset produce un documento. Sus valores numéricos aceptados se almacenan juntos; el original permanece en raw.

Campo específico	Tipo / presencia	Regla
fecha_medicion / fecha_recepcion	date / ambas obligatorias	Momento observado y recepción original; conservar ambas al reprocesar.
tipo	parking | trafico | otro	Coherente con el tipo del sensor. Otros dominios se identifican con sus variables.
intervalo_segundos	int positivo / opcional	Solo cuando la fuente define el intervalo del conteo o promedio.
contexto.id_estacionamiento	long / obligatorio en parking	Prohibido en tráfico y otros dominios. Coincide con fuente y zona.
contexto.id_ruta / id_nivel	long / opcionales en tráfico	Prohibidos en parking y otros dominios. Ruta consistente con fuente y ciudad.
mediciones	array obligatorio, acotado a 64 elementos	Objetos: id_variable long, codigo string, valor decimal, unidad string. ID y unidad canónica coinciden con PostgreSQL.
calidad	valida | parcial | sin_valores_validos	Sin valores válidos implica array vacío; los otros estados requieren al menos una medición aceptada.
errores	array de {campo, motivo} / opcional	Requerido cuando calidad no es valida; registra lo descartado, sin sustituirlo por cero.
raw	object obligatorio	Copia de una fila original. No contiene archivos enteros ni series completas.
trazabilidad	object obligatorio	archivo, fila y version_ingesta; identifican cómo reconstruir la observación.

Decisión de estructura. Se usa un array de mediciones identificado por variable, en lugar de columnas fijas de temperatura u ocupación. Así se corresponde con REGISTRA y con mediciones_limpias. La unidad y el código copiados facilitan las consultas, pero no reemplazan el catálogo autoritativo.

04 / Ejemplos de lecturas

Ejemplos sintéticos completos en notación mongosh: NumberLong, NumberDecimal e ISODate representan tipos BSON. IDs ilustrativos, no datos verificados del CSV.

```
{
 _id: "lec:1:fila1001", schema_version: 1,
 id_fuente: NumberLong("1"),
 clave_origen: "archivoA:fila1001",
 naturaleza: "sintetica",
 procedencia: "simulado",
 id_sensor: NumberLong("12"),
 tipo: "parking",
 fecha_medicion: ISODate("2026-01-01T10:00:00Z"),
 fecha_recepcion: ISODate("2026-01-01T10:01:00Z"),
 contexto: {
  id_ubicacion: NumberLong("3"),
  id_ciudad: NumberLong("1"),
  id_tipo_sensor: NumberLong("2"),
  id_categoria: NumberLong("1"),
  id_estacionamiento: NumberLong("7")
 },
 mediciones: [
  {id_variable: NumberLong("1"),
   codigo: "capacidad", unidad: "plazas",
   valor: NumberDecimal("100")},
  {id_variable: NumberLong("2"),
   codigo: "ocupacion", unidad: "plazas",
   valor: NumberDecimal("95")}
 ],
 calidad: "valida",
 raw: {SystemCodeNumber: "A", Capacity: 100,
       Occupancy: 95, LastUpdated: "2026-01-01 10:00"},
 trazabilidad: {archivo: "parking_simulado.csv",
                fila: 1001, version_ingesta: "1"}
}
```

```
{
 _id: "lec:2:fila2001", schema_version: 1,
 id_fuente: NumberLong("2"),
 clave_origen: "archivoB:fila2001",
 naturaleza: "sintetica",
 procedencia: "simulado",
 id_sensor: NumberLong("25"),
 tipo: "trafico",
 fecha_medicion: ISODate("2026-01-01T10:00:00Z"),
 fecha_recepcion: ISODate("2026-01-01T10:01:00Z"),
 contexto: {
  id_ubicacion: NumberLong("9"),
  id_ciudad: NumberLong("2"),
  id_tipo_sensor: NumberLong("3"),
  id_categoria: NumberLong("1"),
  id_ruta: NumberLong("8"),
  id_nivel: NumberLong("2")
 },
 mediciones: [
  {id_variable: NumberLong("3"),
   codigo: "velocidad_media", unidad: "km/h",
   valor: NumberDecimal("24")}
 ],
 calidad: "valida",
 raw: {Route_ID: "R01", Average_Speed_kmh: 24,
       Timestamp: "2026-01-01 10:00"},
 trazabilidad: {archivo: "trafico_simulado.csv",
                fila: 2001, version_ingesta: "1"}
}
```

Las zonas y ciudades distintas son intencionales: no se presume que ambos datasets sean geográfica o temporalmente compatibles. La tasa de ocupación se calcula sobre capacidad y ocupación.

05 / Eventos, alertas e incidentes

Eventos: hechos detectados. Alertas: avisos disparados por reglas. Incidentes: sucesos reportados; no se infiere un accidente a partir de una velocidad baja.

Colección	Campos específicos obligatorios	Opcionales y semántica
eventos	id_lectura string; tipo_evento string; fecha_evento date; severidad; detector {codigo, version}; evidencia object	Cada evento se deriva de una lectura existente en esta versión del diseño. Un evento no implica una alerta ni un incidente. Para análisis por ventanas, ampliar explícitamente el contrato de evidencia.
alertas	id_evento string; fecha_emision date; severidad; estado; regla {codigo, version, indicador, operador, umbral decimal, unidad}; evidencia object	fecha_cierre date solo al cerrar. Estado: activa, atendida o cerrada. La regla es una copia versionada de la aplicada, no una FK inexistente en PostgreSQL.
incidentes	categoria string; descripcion string; fecha_inicio date; fecha_registro date; estado; detalle object	id_evento string e id_sensor long opcionales. fecha_fin date al resolver. Estado: abierto, en_atencion, resuelto. Puede ser un reporte original o una simulación declarada.

Campos comunes. Estas tres colecciones también incorporan el contrato de la página 2. Severidad: informativo, advertencia o critico. categoria de incidente es una clasificación propia (por ejemplo cierre_vial); no se confunde con id_categoria de sensores. Los catálogos de categorías y tipos de evento pueden comenzar como listas validadas del proyecto.

Referencias. Una lectura tiene 0..N eventos; cada evento tiene exactamente una lectura. Un evento tiene 0..N alertas; cada alerta tiene exactamente un evento. Un incidente referencia 0..1 evento; un evento puede estar vinculado con 0..N incidentes. No se guardan listas de hijos dentro de los padres.

Coherencia. Evento y alerta copian sensor, fuente, naturaleza y contexto de su evidencia. El incidente vinculado debe corresponder al mismo contexto; un reporte sin evidencia conserva su propia fuente. No modificar retrospectivamente esas copias ante un simple cambio de nombre en el catálogo.

06 / Ejemplos de evento y alerta

Ambos ejemplos se derivan de lec:1:fila1001. El 95 % de ocupación es un cálculo; el umbral de 90 % es una regla ilustrativa del proyecto.

```
{
 _id: "evt:1:ocupacion:fila1001:v1",
 schema_version: 1, id_fuente: NumberLong("1"),
 clave_origen: "evt:ocupacion:fila1001:v1",
 naturaleza: "sintetica", procedencia: "derivado",
 id_sensor: NumberLong("12"),
 id_lectura: "lec:1:fila1001",
 contexto: {
  id_ubicacion: NumberLong("3"),
  id_ciudad: NumberLong("1"),
  id_tipo_sensor: NumberLong("2"),
  id_categoria: NumberLong("1"),
  id_estacionamiento: NumberLong("7")
 },
 tipo_evento: "ocupacion_alta",
 fecha_evento: ISODate("2026-01-01T10:00:00Z"),
 severidad: "advertencia",
 detector: {codigo: "ocupacion_alta", version: "1"},
 evidencia: {indicador: "tasa_ocupacion",
             valor: NumberDecimal("95"),
             unidad: "%"}
}
```

```
{
 _id: "ale:1:ocupacion:fila1001:v1",
 schema_version: 1, id_fuente: NumberLong("1"),
 clave_origen: "ale:ocupacion:fila1001:v1",
 naturaleza: "sintetica", procedencia: "derivado",
 id_sensor: NumberLong("12"),
 id_evento: "evt:1:ocupacion:fila1001:v1",
 contexto: {
  id_ubicacion: NumberLong("3"),
  id_ciudad: NumberLong("1"),
  id_tipo_sensor: NumberLong("2"),
  id_categoria: NumberLong("1"),
  id_estacionamiento: NumberLong("7")
 },
 fecha_emision: ISODate("2026-01-01T10:01:01Z"),
 severidad: "advertencia", estado: "activa",
 regla: {codigo: "ocupacion_90", version: "1",
         indicador: "tasa_ocupacion",
         operador: ">", umbral: NumberDecimal("90"),
         unidad: "%"},
 evidencia: {valor: NumberDecimal("95")}
}
```

La clave de deduplicación debe construirse con el ID completo de evidencia y el código/versión de la regla; los textos de estos ejemplos son abreviados legibles. Reprocesar no crea otro aviso para el mismo hecho y regla.

Una nueva versión de la regla puede producir otro resultado identificado por su versión. El evento registra el instante observado y la alerta el instante de emisión; no se exige que ambos instantes sean iguales.

07 / Ejemplo de incidente y decisiones de datos

Un incidente puede existir sin evento ni sensor asociados. No se obliga a inventar esas referencias para un reporte urbano.

```
{
 _id: "inc:3:reporte42", schema_version: 1,
 id_fuente: NumberLong("3"),
 clave_origen: "reporte42",
 naturaleza: "sintetica", procedencia: "simulado",
 contexto: {
  id_ubicacion: NumberLong("3"),
  id_ciudad: NumberLong("1")
 },
 categoria: "cierre_estacionamiento",
 descripcion: "Cierre temporal por mantenimiento",
 fecha_inicio: ISODate("2026-01-01T12:00:00Z"),
 fecha_registro: ISODate("2026-01-01T11:00:00Z"),
 estado: "abierto",
 detalle: {id_estacionamiento: NumberLong("7"),
           motivo: "Mantenimiento simulado"}
}
```

Fuente 3. Debe existir en PostgreSQL como fuente de simulación de incidentes. El ejemplo no afirma que el dataset de parking contenga cierres.

Campos ausentes. Si un vínculo opcional no se conoce, se omite. id_organismo no se rellena con una institución ficticia. id_sensor se añade solo si está identificado.

Resolución. Al cambiar estado a resuelto se exige fecha_fin, no anterior a fecha_inicio. La fecha de registro puede anteceder al inicio en incidentes programados.

GeoJSON opcional. Si existen coordenadas fiables, contexto.geo = {type: "Point", coordinates: [longitud, latitud]}. Se debe señalar si es la ubicación del sensor o un punto representativo de zona; no atribuir precisión inexistente.

Histórico y tamaño. Cada documento representa un hecho, no toda la vida del sensor. El diseño establece un límite operativo de 256 KiB por documento y 64 mediciones por lectura; filas que excedan el contrato se apartan con su motivo y se conserva el original en HDFS. No se borran automáticamente los registros necesarios para la defensa.

Datos no válidos. Si se conocen sensor y fecha, se conserva la lectura con raw y errores aunque no tenga valores válidos. Si faltan identificadores o tiempo esenciales, la fila queda en una salida de rechazos del proceso de ingesta, no se fuerza al esquema de lecturas.

08 / Validaciones y consistencia

El documento define el contrato. Los validadores e índices se implementarán después; no se han ejecutado pruebas contra una instancia MongoDB.

Nivel	Reglas a implementar
Validador de colección ($jsonSchema)	Campos obligatorios, tipos BSON, enums, formatos de objetos, tamaño máximo del array y campos permitidos en el núcleo. Permitir flexibilidad dentro de raw, detalle y evidencia; rechazar propiedades inesperadas en el resto.
Expresiones del validador ($expr)	Unicidad de id_variable dentro del array mediante comparación de tamaño con el conjunto de IDs; calidad consistente con longitud del array; fechas de cierre/fin coherentes. Un índice multikey unique no resuelve por sí solo la repetición dentro de un documento.
Reglas de dominio	Mediciones numéricas finitas; ocupación y capacidad enteras no negativas; ocupación ≤ capacidad cuando ambas existen; no dividir entre cero. Validar contextos exclusivos parking/tráfico.
Catálogos PostgreSQL	Verificar existencia de IDs, variable permitida para tipo, unidad, código y coherencia de zona/ciudad/fuente/ruta. Requiere ingesta o servicio: un validador local no consulta PostgreSQL.
Referencias MongoDB	Comprobar lectura antes de evento y evento antes de alerta; validar un id_evento opcional en incidente. Las referencias no son FK automáticas.
Reprocesamiento	Mismo registro conserva _id, clave_origen y fecha_recepcion original. Upsert por clave única; reemplazar conjuntamente mediciones/calidad/errores/contexto si se corrige un registro, con política de corrección documentada.
Borrados y cambios	No borrar una lectura referenciada ni un evento con alertas/incidentes sin resolver primero esas dependencias. Demostrar deleteOne con registros de prueba sin referencias.
Publicación	Procesar en orden y con claves estables; reintentar fallos sin duplicar. Evitar publicar un lote incompleto en PostgreSQL. No se presupone una transacción atómica entre motores.

09 / Índices propuestos

Son índices iniciales asociados a consultas concretas; confirmar su utilidad con explain y el volumen real. _id ya tiene su índice único.

Colección	Índice	Propósito
Todas	{id_fuente:1, clave_origen:1} UNIQUE	Idempotencia por fuente y registro, en cada colección.
lecturas	{id_sensor:1, fecha_medicion:-1}	Histórico de un sensor.
lecturas	{fecha_medicion:1}	Rangos de fechas sin filtrar sensor.
lecturas	{"contexto.id_ubicacion":1, fecha_medicion:1}	Consultas por zona y periodo.
lecturas	{"contexto.id_tipo_sensor":1, fecha_medicion:1}	Consultas por tipo y periodo.
lecturas	{"mediciones.id_variable":1, fecha_medicion:1}	Filtro por variable. Usar $elemMatch cuando varios filtros deban coincidir en la misma medición.
eventos	{severidad:1, fecha_evento:-1}; {id_lectura:1}	Eventos críticos y búsqueda de evidencia.
alertas	{"contexto.id_ubicacion":1, fecha_emision:-1}; {id_evento:1}	Alertas por zona y trazabilidad.
incidentes	{categoria:1, fecha_inicio:-1}; {id_evento:1}	Clasificación de incidentes y vínculo opcional.
Opcional	{"contexto.geo":"2dsphere"}	Solo para búsquedas espaciales; el mapa no obliga a tener este índice.

Colecciones normales. Se eligen para facilitar las operaciones CRUD exigidas y la deduplicación mediante índices únicos. No se requiere usar colecciones time series en esta primera versión; su comportamiento y restricciones deben revisarse antes de migrar. No se configura TTL para no perder evidencia histórica.

10 / Cobertura de las consultas obligatorias

La estructura soporta estas consultas sin acceder a PostgreSQL en cada agregación. La existencia de datos suficientes se verifica durante la carga.

N.º / consulta	Colección y campos	Operación prevista
1. Lecturas de un sensor	lecturas.id_sensor	find con filtro e índice sensor-fecha.
2. Lecturas entre fechas	lecturas.fecha_medicion	find con $gte y $lt.
3. Máximos y mínimos	mediciones.id_variable, valor	Filtrar periodo; $unwind; filtrar variable; $group con $max y $min.
4. Eventos críticos	eventos.severidad, fecha_evento	find con severidad = critico y periodo.
5. Alertas por zona	alertas.contexto.id_ubicacion	Filtrar fecha; $group por zona y $sum:1.
6. Promedio por zona	lecturas.contexto, mediciones	Agrupar por zona + variable; $avg sobre valores aceptados en unidad canónica.
7. Promedio por tipo	contexto.id_tipo_sensor, mediciones	Agrupar por tipo + variable; no mezclar capacidad, velocidad y temperatura.
8. Incidentes por categoría	incidentes.categoria	Filtrar periodo; $group por categoría y conteo.
9. Agregaciones	lecturas.id_sensor y mediciones	Contar lecturas por sensor; contar mediciones como suma del tamaño del array. Son métricas diferentes.
10. Filtros múltiples	sensor, zona, fecha y mediciones	Combinar filtros; $elemMatch para variable y valor del mismo elemento.

Consultas adicionales. Evolución de una variable: agrupar por intervalo temporal. Umbrales superados: consultar alertas con regla y evidencia. Ocupación por zona: definir si se busca promedio por observación o suma de ocupaciones / suma de capacidades en un mismo instante; no sumar todo el histórico como vehículos únicos.

11 / Integración con el modelo PostgreSQL

El diseño conserva el significado de SENSOR, UBICACIÓN, FUENTE, VARIABLE y REGISTRA del modelo conceptual revisado.

MongoDB	PostgreSQL / proceso	Regla de correspondencia
lecturas._id	lecturas_limpias.id_documento_mongo	Referencia textual única. No convertirla en id_lectura numérico.
id_sensor / id_fuente / clave_origen	lecturas_limpias	Mismos IDs y clave estable del registro original.
fecha_medicion / fecha_recepcion / calidad	lecturas_limpias	No sustituir fechas observadas por fecha de carga de Spark.
mediciones[]	mediciones_limpias + variables_medicion	Un elemento por pareja lectura-variable; valor decimal → numeric.
contexto.id_estacionamiento	lecturas_parking	Solo para parking.
contexto.id_ruta / id_nivel	lecturas_trafico	Solo para tráfico; referencias opcionales.
eventos / alertas / incidentes	MongoDB + exportación histórica HDFS	No requieren nuevas tablas PostgreSQL para existir; el dashboard puede consultarlos directamente.

Criterios de aceptación. Repetir una carga no aumenta documentos; duplicar una variable se rechaza; contexto incompatible se rechaza; una alerta encuentra su evento y su lectura; los números long conservan los IDs; la copia de contexto coincide con el catálogo al capturarla; calidad e inconsistencias quedan visibles. El equipo debe demostrar al menos 500 000 documentos, no 500 000 elementos de arrays.

Alcance pendiente. Parking y tráfico no acreditan temperatura, contaminación, energía o incidentes reales. Se requieren fuentes adicionales compatibles o una adaptación del alcance aprobada. Los ejemplos sintéticos de este documento no prueban volumen, cobertura temporal ni contenido de los CSV.

Referencias técnicas: mongodb.com/docs/manual/data-modeling/embedding/ · mongodb.com/docs/manual/tutorial/unique-indexes-schema-validation/ · mongodb.com/docs/manual/core/timeseries/timeseries-limitations/. Consultadas el 9 octubre 2026. Base del proyecto: enunciado SmartCity Analytics, pp. 8-12 y 26; modelo conceptual PostgreSQL revisado.